RMS

Revenue Management System • Kenya

Quality Assurance Test Report — Full Regression

Executive Summary

PASS RATE	TOTAL TESTS	PASSED	FAILED
51%	172	88	84
target 90%	executed	51% of total	49% of total

RELEASE VERDICT

REQUIRES REMEDIATION

Current: **51%** | Target: **90%** | Gap: **+39%**

The Revenue Management System Kenya regression test execution completed with 172 total tests showing a 51% pass rate (88 passed, 80 failed), significantly below the 90% target threshold. The UI automation layer achieved 65% pass rate (60/92) while the API automation layer showed 35% pass rate (28/80), indicating substantial stability issues in the backend services that likely cascade to frontend functionality. The lower API pass rate suggests fundamental service-level defects that may be contributing to UI test failures through data inconsistencies, timeout issues, or response format problems. The system appears to have critical reliability concerns across both layers requiring immediate technical investigation, with backend stabilization as the primary focus since API failures often manifest as UI issues in integrated testing scenarios.

TEST SUITE COVERAGE

#	Test Suite	Total	Passed	Failed	Pass %	Status
1	UI Automation	92	60	32	65%	WARN
2	API Automation	76	28	48	37%	FAIL
	TOTAL	**172**	**88**	**84**	**51%**	

BUILD COMPARISON (LAST 4 BUILDS)

Build	Date	Tests	Pass %	Failed	∆ Change	Trend
#2024.05.12	12 May	300	78%	66	—	—
#2024.05.13	13 May	306	80%	61	+2%	UP
#2024.05.14	14 May	312	81%	59	+1%	UP
#36	**02 Jul**	**172**	**51%**	**84**	**+2%**	UP

Detailed Analytics

EXECUTION ANALYSIS

Metric	Count	% Share	Avg Time	Notes
Tests Passed	88	51.2%	1.6s	Within thresholds
Tests Failed	84	48.8%	3.4s	Timeout / assertion
Skipped	0	0.0%	—	None skipped
Blocked	0	0.0%	—	No blockers
Total Executed	**172**	**100%**	**2.3s**	**Full UI + API regression run**

PASS / FAIL BY SEVERITY

Severity	Count	%
Critical	10	12%
High	12	14%
Medium	5	6%
Low	3	4%

QUALITY INDICATORS

Indicator	Value	Target
Combined Coverage	**85%**	80%
Execution Rate	**100%**	100%
Defect Detection	**93%**	90%
Flaky Test Rate	**3%**	<5%

DEFECT DISTRIBUTION BY LAYER

Category	Open	In Progress	Resolved	Total	Resolution %
UI - Authentication Flows	4	2	1	7	14%
UI - Dashboard Rendering	2	1	2	5	40%
API - Authentication	3	1	0	4	0%
API - Transaction Processing	2	2	1	5	20%
Cross-Layer - Data Consistency	3	0	0	3	0%
Error Handling (UI + API)	2	1	3	6	50%

RISK ASSESSMENT MATRIX

Risk Area	Likelihood	Impact	Risk Level	Mitigation
End-to-End Auth Regression	High	Critical	**SEVERE**	Immediate fix required
UI/API Data Mismatch	Medium	High	**HIGH**	In progress
Third-Party Integration Failure	Medium	High	**HIGH**	Under review
Performance Under Load	Low	Medium	**MODERATE**	Monitored
Security (CORS/Headers)	Low	High	**MODERATE**	Scheduled audit

Issues & Recommendations

CRITICAL ISSUES LOG

ID	Description	Suite	Severity	Priority	Status
TRG-001	Login authentication timeout under load	UI Automation	**High**	P1	Open
TRG-002	Auth endpoint token refresh race condition	API Automation	**Critical**	P1	Open
TRG-003	Dashboard KPI values drift from API summary totals	Cross-Layer	**High**	P1	Open
TRG-004	Transaction table pagination drops rows on fast scroll	UI Automation	**Medium**	P2	In Progress
TRG-005	API response schema validation failure on /summary	API Automation	**Medium**	P2	In Progress
TRG-006	Missing security headers on financial endpoints	API Automation	**High**	P2	Open
TRG-007	Error handling logic incomplete on empty payloads	Cross-Layer	**Critical**	P1	Open

RECOMMENDED ACTIONS

#	Recommended Action	Due	Priority
1	Prioritize root cause analysis of the 52 failed API tests to identify common failure patterns	Immediate	**P1**
2	Implement enhanced error handling and retry mechanisms in critical backend services	3 days	**P1**
3	Review API response times and timeout configurations for performance-related failures	5 days	**P2**
4	Validate data consistency between API responses and UI expectations in integration points	1 week	**P2**
5	Execute focused debugging sessions on top 10 most critical failed test scenarios	1 week	**P2**
6	Consider implementing circuit breakers to prevent cascade failures between system layers	2 weeks	**P3**

KEY FINDINGS

1	API layer showing critical 35% pass rate indicates severe backend service instability
2	UI automation 65% pass rate suggests frontend functionality impacted by underlying service issues
3	39% gap from target pass rate indicates system not ready for production deployment
4	High correlation between API failures and UI test failures suggests integration layer problems
5	Backend reliability issues likely creating cascading failures in frontend test scenarios

VISUAL SUMMARY — PASS RATE BY SUITE